

VCGen

Boogie Style

Idea

Boogie-style VCGen uses backward safety equations over the CFG.

For each block i , introduce:

$$\text{ok}_i$$

ok_i means: starting execution at block i is safe.

The counterexample query is:

$$\neg \text{ok}_{\text{entry}}$$

Block Equation

$$\text{ok}_i \Leftrightarrow \left(\text{defs}_i \Rightarrow \left(\text{asserts}_i \wedge \bigwedge_{j \in \text{succ}(i)} \left((\text{cond}_{i,j} \wedge \text{dsa}_{i,j}) \Rightarrow \text{ok}_j \right) \right) \right)$$

- assumptions and definitions are premises
- assertions are consequents
- each feasible successor must be safe
- infeasible blocks are safe vacuously

Terminal Block

For a terminal block, the successor conjunction is true:

$$ok_i \Leftrightarrow (defs_i \Rightarrow asserts_i)$$

A false reachable assertion makes the local `ok_i` false, then unsafety propagates backward to entry.

DSA Diamond

entry:

x := havoc

y := havoc

c := x < y

if c goto left else right

left:

a_left := x + 1

a := a_left

goto join

right:

a_right := y + 1

a := a_right

goto join

DSA Diamond: Exit

```
join:
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

The assignments to `a` are edge-sensitive DSA definitions for the incoming edges to `join`.

Encoded Equations

$$\mathbf{OK}_{\text{entry}} : \text{ok}_{\text{entry}} \Leftrightarrow (c = (x < y) \Rightarrow ((c \Rightarrow \text{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \text{ok}_{\text{right}})))$$

$$\mathbf{OK}_{\text{left}} : \text{ok}_{\text{left}} \Leftrightarrow (a_{\text{left}} = x + 1 \Rightarrow ((a = a_{\text{left}}) \Rightarrow \text{ok}_{\text{join}}))$$

$$\mathbf{OK}_{\text{right}} : \text{ok}_{\text{right}} \Leftrightarrow (a_{\text{right}} = y + 1 \Rightarrow ((a = a_{\text{right}}) \Rightarrow \text{ok}_{\text{join}}))$$

Encoded Exit

$$\mathbf{OK}_{\text{join}} : \text{ok}_{\text{join}} \Leftrightarrow (\text{ok} = (a > 0) \Rightarrow \text{ok}_{\text{exit}})$$

$$\mathbf{OK}_{\text{exit}} : \text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$$

$$\mathbf{DISCHARGE} : \neg \text{ok}_{\text{entry}}$$

The model chooses havoc values and a branch whose successor is unsafe.

Static ITE Merge Variant

The merge can be compiled into an ordinary definition:

```
join:  
  a := ite(c, a_left, a_right)  
  ok := a > 0  
  goto exit
```

$$\text{ok}_{\text{join}} \Leftrightarrow \left((a = \text{ite}(c, a_{\text{left}}, a_{\text{right}}) \wedge \text{ok} = (a > 0)) \Rightarrow \text{ok}_{\text{exit}} \right)$$

Tradeoffs

Advantages:

- no separate path-selection formula
- many assertions are natural
- assumptions are handled by implication

Costs:

- formula is nested
- cross-block simplification is harder
- ϕ /ITE extensions need exclusivity side conditions

Phi Extension Needs CFG Facts

If a phi is compiled as:

$$a = \text{ite}(\text{bb}_{\text{left}}, a_{\text{left}}, a_{\text{right}})$$

then the block variables used by the ITE must be consistent:

$$\begin{aligned} \text{bb}_{\text{join}} &\Rightarrow \text{bb}_{\text{left}} \vee \text{bb}_{\text{right}} \\ &\neg(\text{bb}_{\text{left}} \wedge \text{bb}_{\text{right}}) \end{aligned}$$

The backward equations prove safety; the CFG facts keep the ITE merge faithful.